Western Norway University of Applied Sciences

Beyond Vibe Coding

From Concept to MVP

Spec-driven, AI-assisted Software Engineering



Asst. Prof. Per Christian Engdal
Western Norway University of Applied Sciences
2026

Agenda

01	**From Vibe Coding To Spec-Driven Development** Why a plausible app is not a finished app, and why specs address it	**Clip · vibe coding**
02	**Agentic Development: Two loops** The product loop and the execution loop	
03	**Agentic Development: Harness and engineering control** Rules, permissions and checks in the repo	
04	**Agentic Development: The product loop** From requirements and evidence, to ready specs and user stories	**Demo 1 · discovery to framing** **Demo 2 · story refinement**
05	**Agentic Development: The execution loop** Building the MVP - from a ready story to an accepted increment	**Demo 3 · plan, build and verify** **Demo 4 · feedback to merge**
06	**Agentic Development: The MVP playbook** Seven steps and the harness you take home, with Lovable as an option	

From Vibe Coding To Spec-Driven Development

Agentic Development

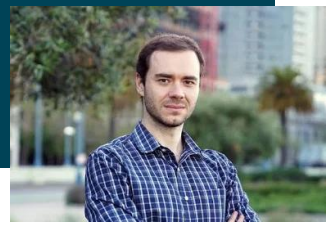
01

What Vibe Coding Means

THE ORIGINAL MEANING

“Fully give in to the vibes, embrace exponentials, and forget that the code even exists.”

Andrej Karpathy, February 2025



IN PRACTICE

Describe it
Tell the model in plain language what you want

Try it
Judge the result by running it, not by reading it

Ask for changes
Repeat until it looks right

Vibe coding means judging the result on screen. Nobody reads the code.

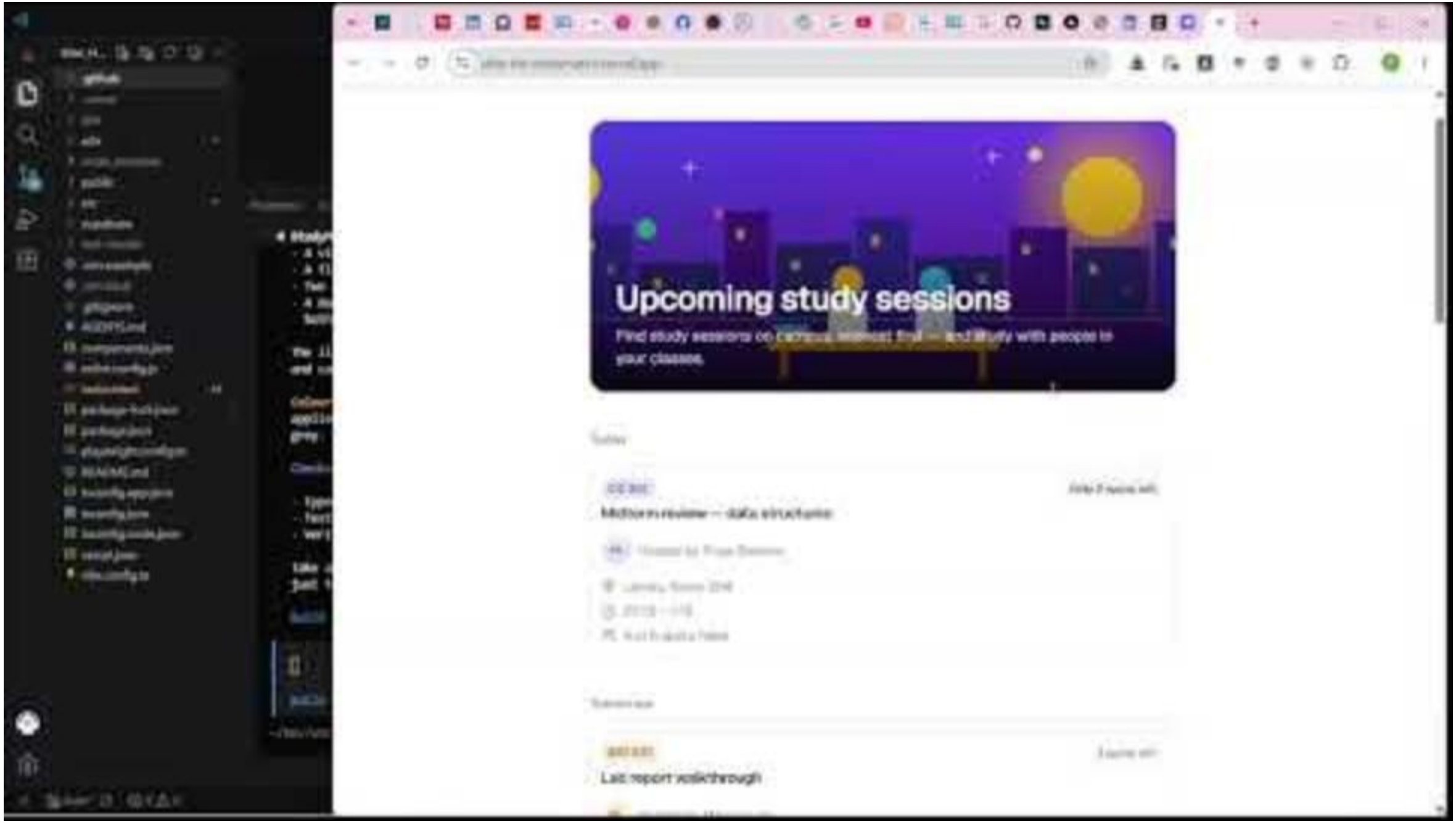
Source: Andrej Karpathy, post on X, 2 February 2025

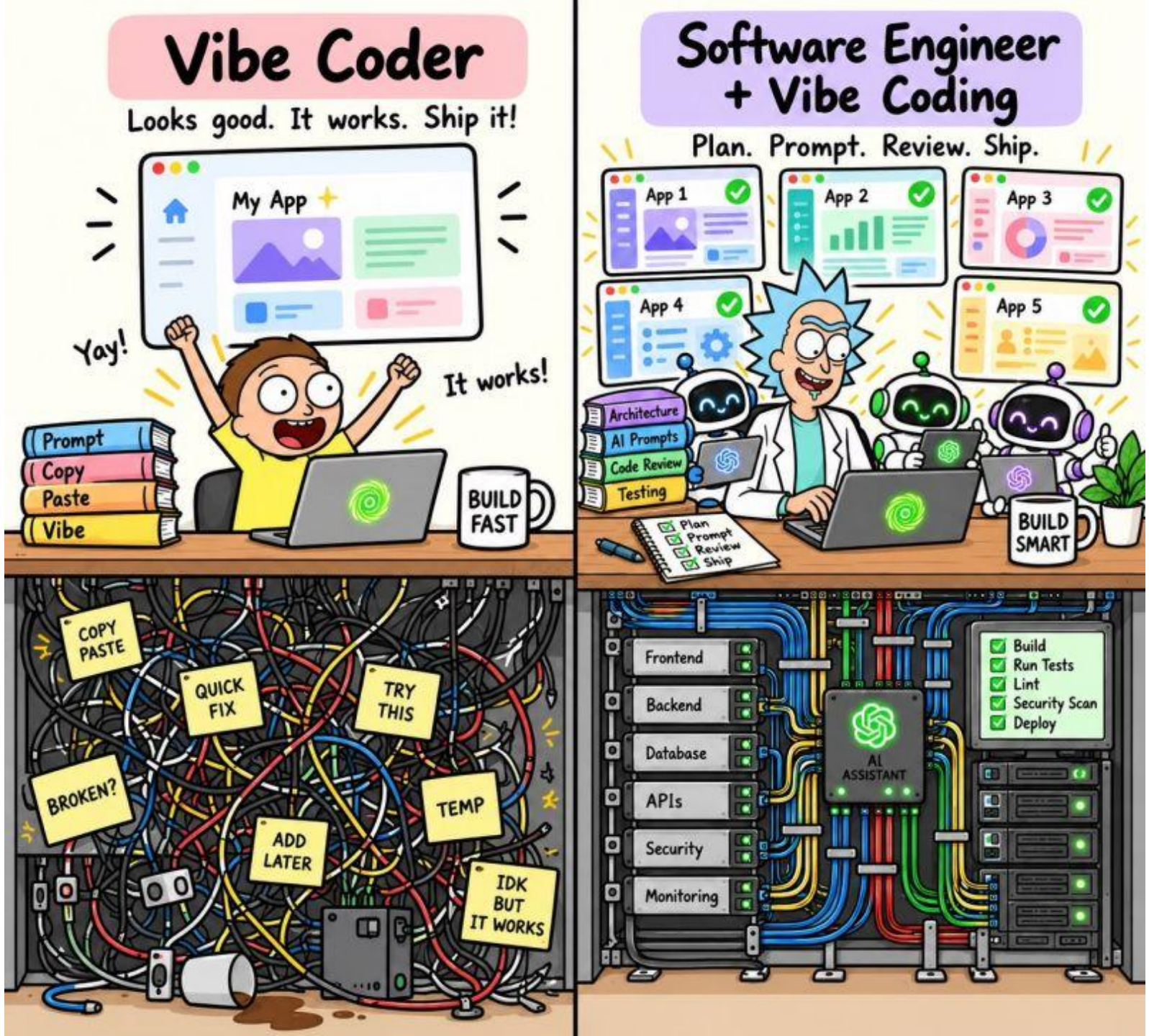
Clip: Vibe Coding

StudyMatch, prompt by prompt

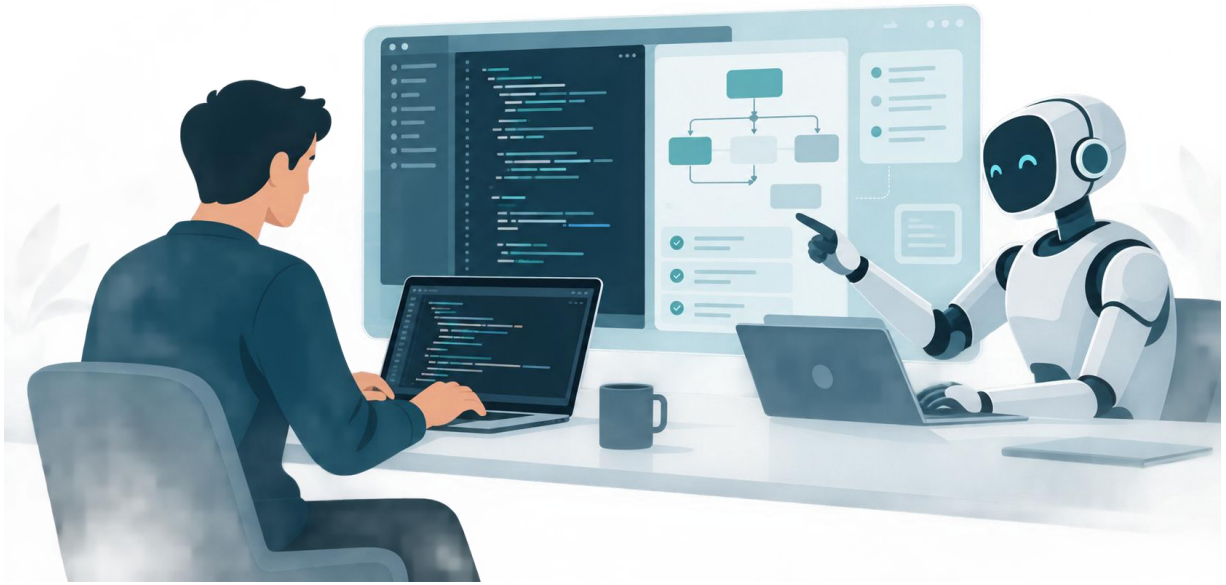
Watch: fast, but nothing is written down or tested




Upcoming study sessions


Vibe Coder
Looks good. It works. Ship it!
My App
Yay!
It works!
Prompt
Copy
Paste
Vibe
BUILD FAST
COPY PASTE
QUICK FIX
TRY THIS
BROKEN?
TEMP
ADD LATER
IDK BUT IT WORKS
Software Engineer + Vibe Coding
Plan. Prompt. Review. Ship.
App 1
App 2
App 3
App 4
App 5
Architecture
AI Prompts
Code Review
Testing
Plan
Prompt
Review
Ship
BUILD SMART
Frontend
Backend
Database
APIs
Security
Monitoring
AI ASSISTANT
Build
Run Tests
Lint
Security Scan
Deploy

With An Agent, You Get A New Coworker



WHAT YOUR NEW COWORKER BRINGS

Knows more
Broad knowledge across every domain

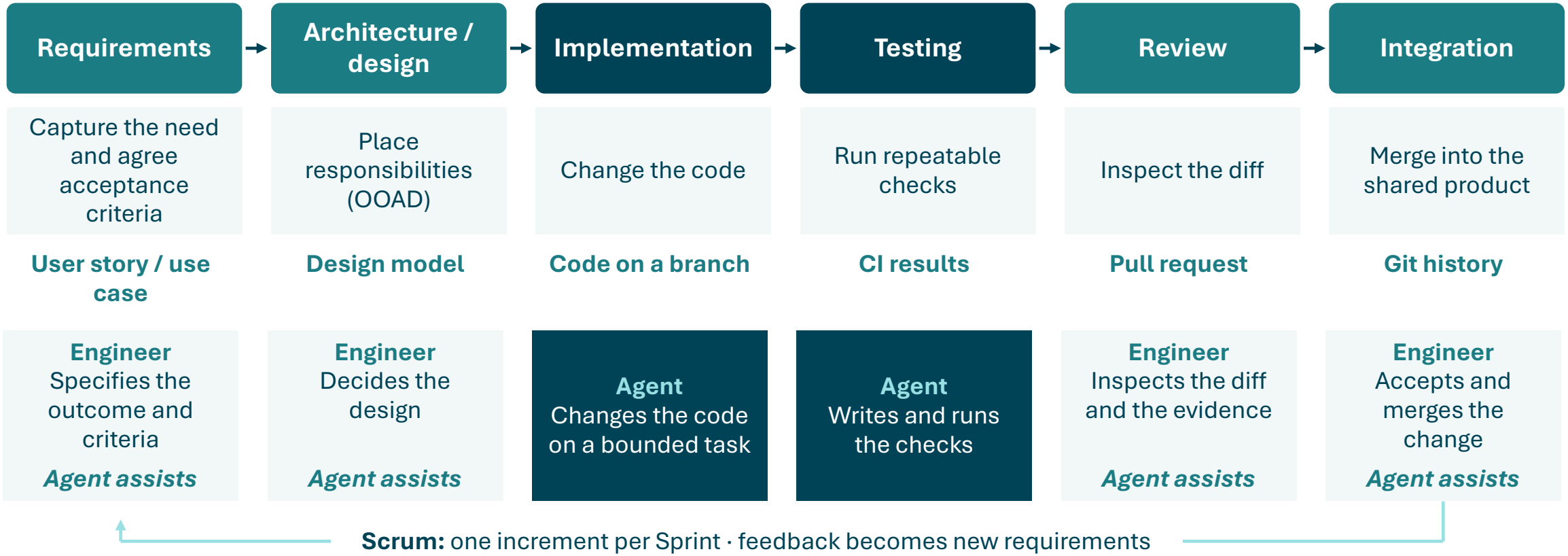
Has seen more
More code and cases than one person sees

Has the tools
Searches, writes, runs and tests in minutes

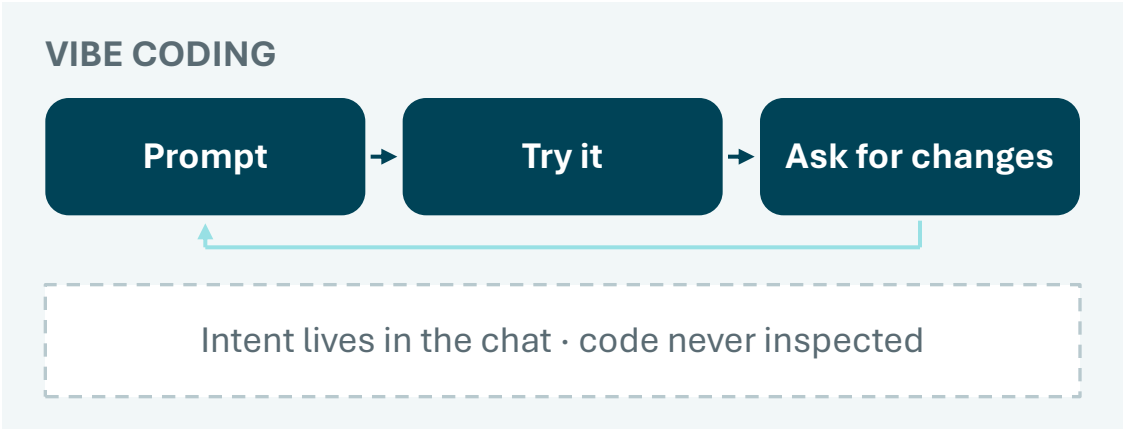
Gets better
High quality today, and each model is better

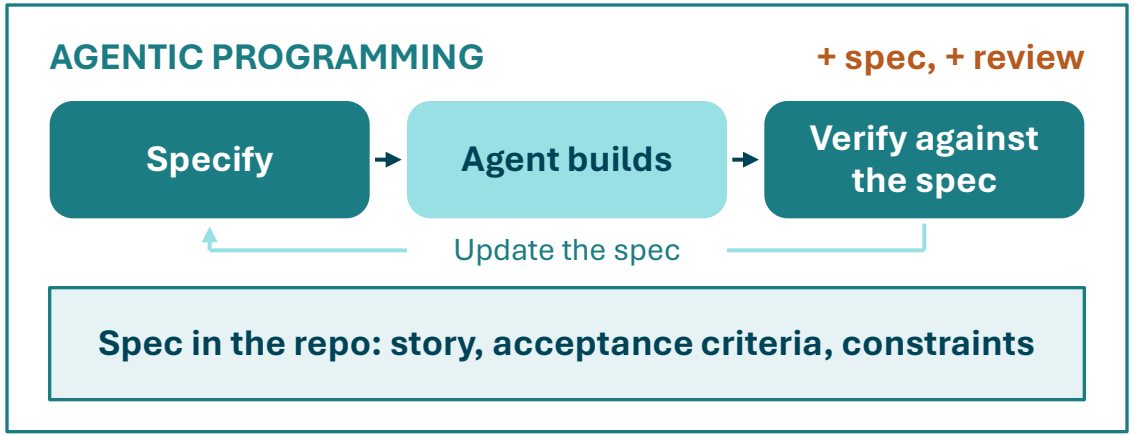
The question is no longer what the agent can do. It is what you ask it to do.

The Software Engineering Workflow - with Agents



Vibe Coding And Agentic Programming





Anyone can do it, even without programming

Each session starts from scratch. Often weak on maintainability, correctness and security.

Needs engineering knowledge

The spec carries intent across sessions and agents. The engineer verifies structure, correctness and security.

When is vibe coding fine?
When all three hold:

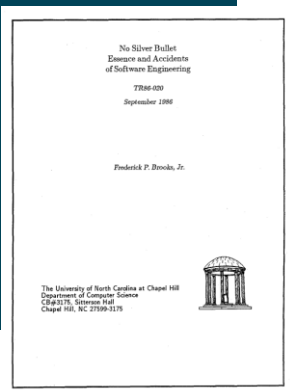
- Throwaway
- Low stakes
- Errors show on use

Specs Give Us Control Over The Agent

A WELL-KNOWN LESSON

“The hardest single part of building a software system is deciding precisely what to build.”

Fred Brooks, No Silver Bullet, 1987


No Silver Bullet
Essence and Accidents
of Software Engineering
TR86-020
September 1986
Frederick P. Brooks, Jr.
The University of North Carolina at Chapel Hill
Department of Computer Science
CB#3175, Sitterson Hall
Chapel Hill, NC 27599-3175

WHAT THE SPEC GIVES US WITH AGENTS

Same prompt, different code
Determinism: code varies, the criteria do not

The agent acts on its own
Control: scope and constraints set the limits

No criteria, no done
Verifiable: the criteria give PASS or FAIL

Faster than you can read
Review: check against the spec, not every line

The agent varies. The spec and its checks do not. That is where the control is.

Source: Frederick P. Brooks, “No Silver Bullet: Essence and Accidents of Software Engineering”, 1987

What We Mean By A Spec

WHAT A SPEC CONTAINS

User story
Who needs what, and why

Acceptance criteria
Testable conditions: PASS or FAIL

Constraints
Security, privacy and compliance rules to meet

Out of scope
What the agent must not change

EXAMPLE **specs/login-lockout.md**

Story
As a user, I want my account locked after five failed logins, so attackers cannot guess my password.

Acceptance criteria
Five failed attempts lock the account for 15 minutes
A successful login resets the counter

Constraints Every lockout is logged for audit

Out of scope Password reset flow

A spec feeds every prompt, but it lives in the repo and outlives the chat.

Architecture and tech choices go in **plan.md**

Two Files: What To Build, Then How

spec.md What and why

CONTAINS
Story, acceptance criteria, constraints, scope

REVIEWED BY
Product owner and engineer

CHANGES WHEN
The need changes

THE AGENT USES IT TO
Know when the work is done

→

plan.md How

CONTAINS
Architecture, tech stack, data model, interfaces

REVIEWED BY
Engineer and tech lead

CHANGES WHEN
The design or technology changes

THE AGENT USES IT TO
Know how to build it

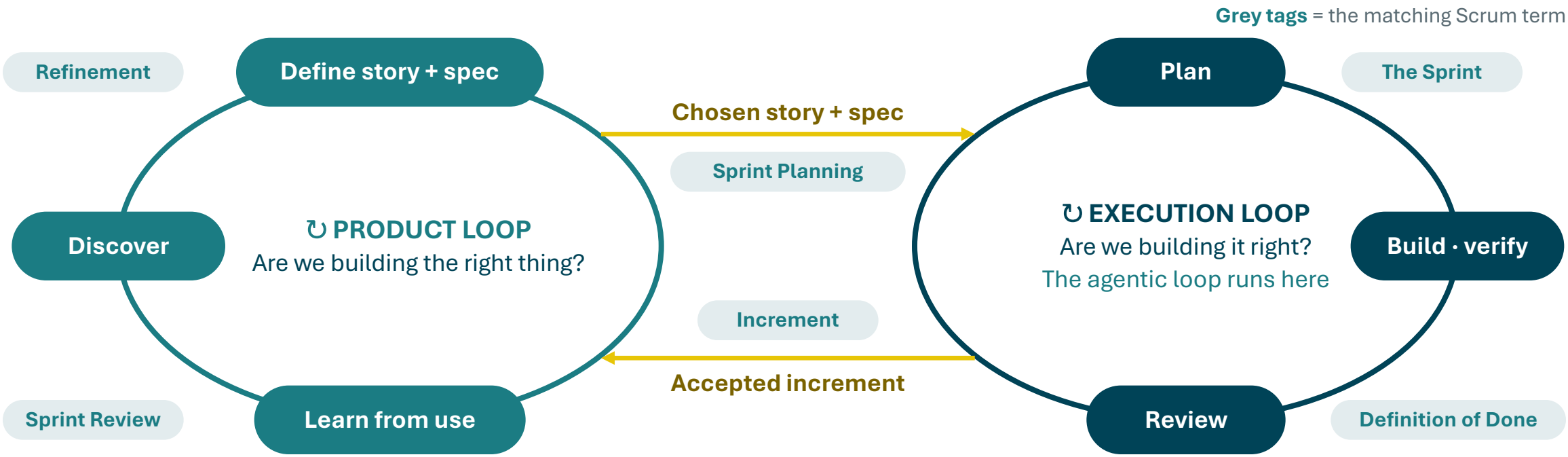
One reason to change per file. Swap the tech without rewriting the need.

Agentic Development: - Two Loops

Agentic Development

02

The Product Loop And The Execution Loop



Led by people: users and the Product Owner
They decide what to build and accept the value

Led by the System Engineer: Planner and Coder do the work
Approves the plan, owns the gate, merges

Agents speed up the execution loop. Only people using the product can close the product loop.

The Product Owner And The System Engineer

PRODUCT OWNER · PRODUCT LOOP

Owns the what and the why

- Vision, scope and priority
- Stories and acceptance criteria
- Answers refinement questions
- Accepts the increment in the Preview
- Decides on feedback

Ready story →

← Accepted increment

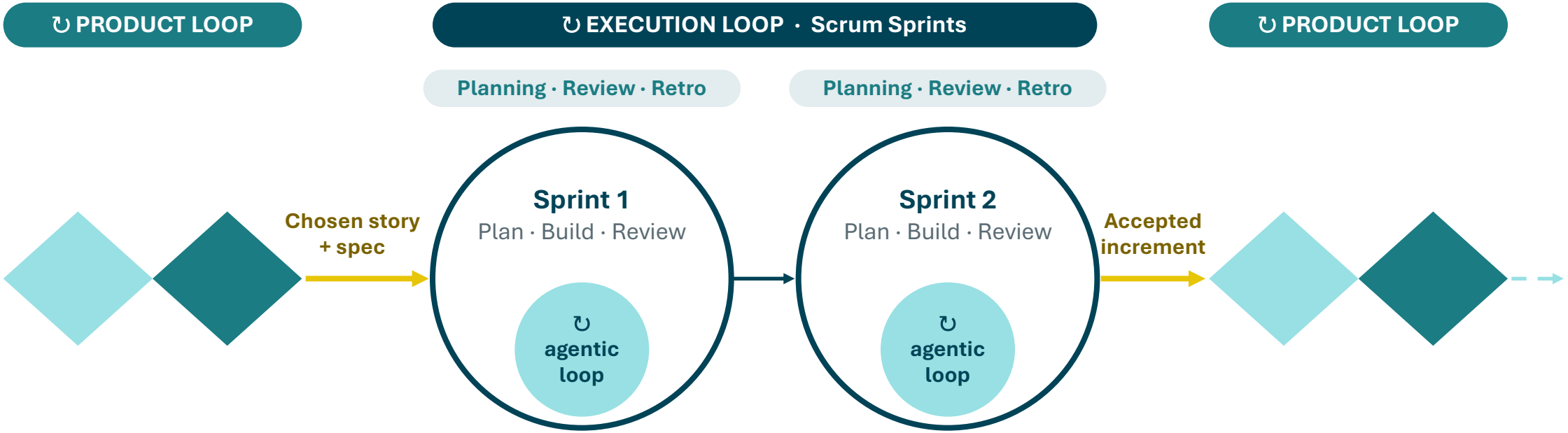
SYSTEM ENGINEER · EXECUTION LOOP

Owns the how and the quality

- Approves the Planner's plan
- Steers the Coder, stops drift
- Owns the gate and Definition of Done
- Reviews the code in the pull request
- Merges and deploys after acceptance

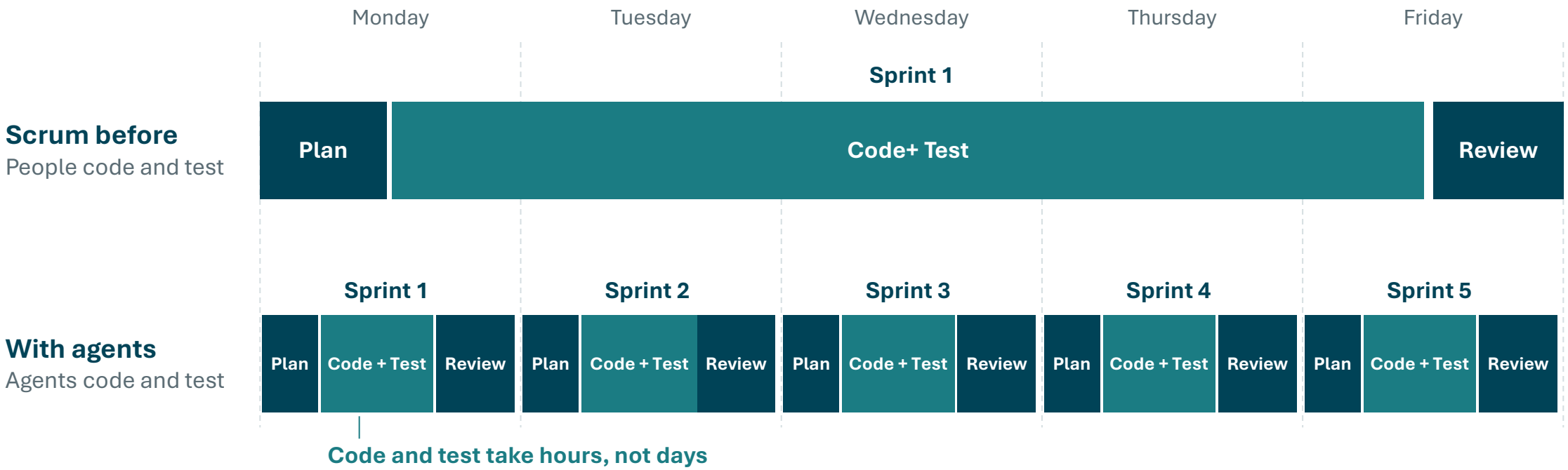
Two roles, as in Scrum. In this course, you play both: decide as PO, build and merge as System Engineer.

Design Thinking Finds What - Scrum Builds It



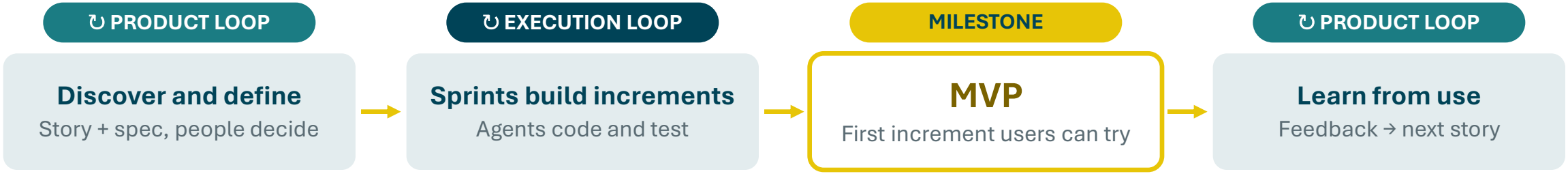
Design Thinking decides what to build. Scrum builds it in increments. Agents work inside each Sprint.

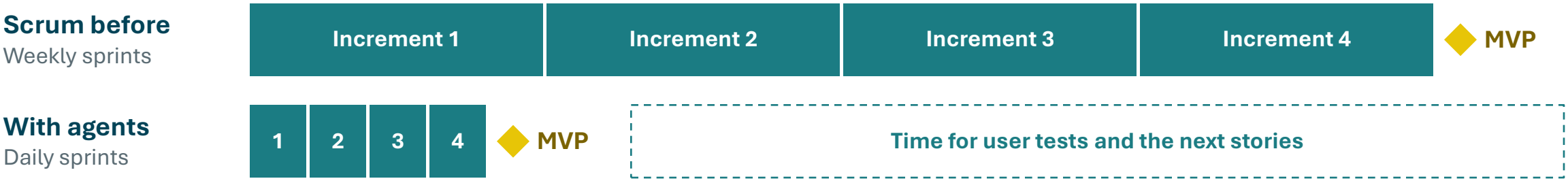
With Agents, Weekly Sprints Can Become Daily



Same week. Agents shorten every sprint, so we run five times as many and ship more.

With Agents, We Can Reach The MVP Sooner





The MVP closes the first execution loop. Agents give us the capacity to get there faster.

Agentic Development - Harness And Engineering Control

Agentic Development

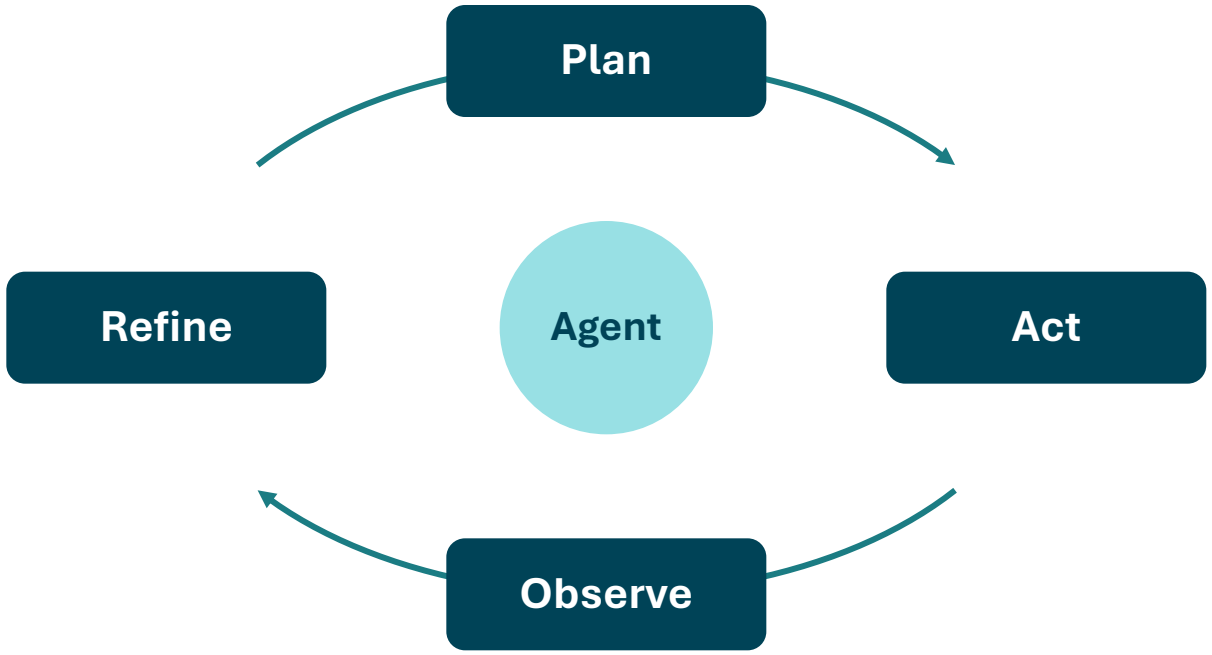
03

How An Agent Works

WHAT GOES IN

Prompt
The task, stated in words

Context
Code, specs and earlier decisions



WHAT IT CAN USE

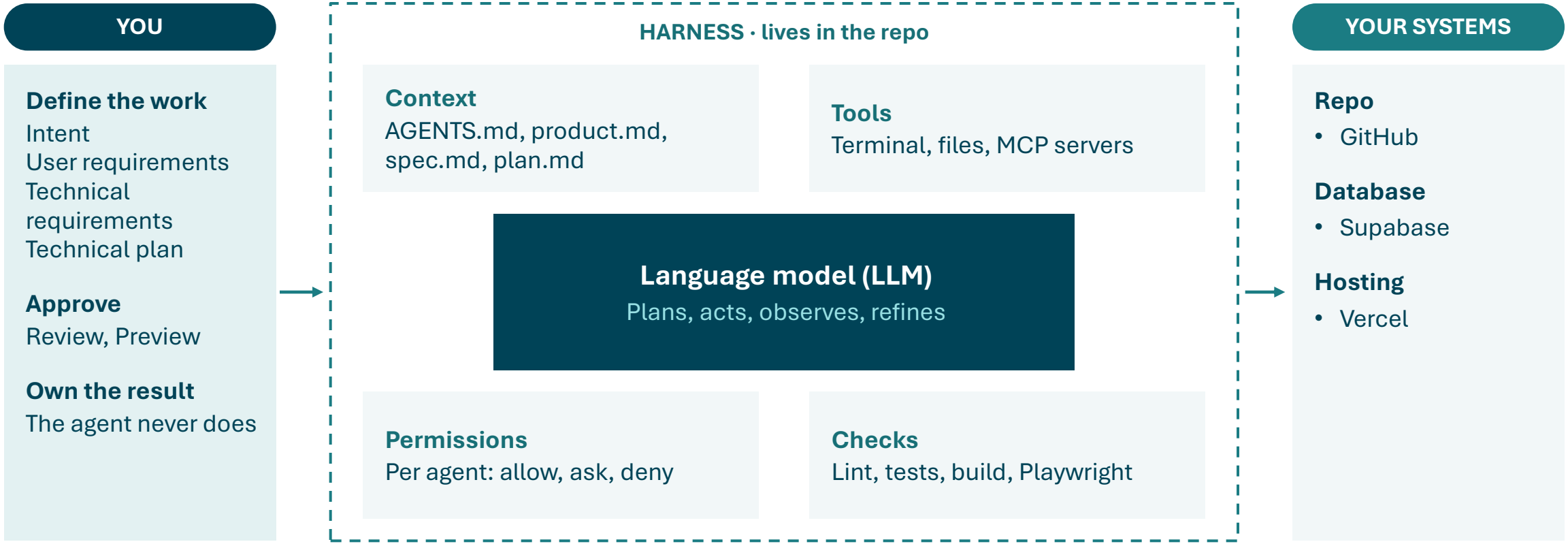
Tools
Run tests, edit files, call APIs (e.g. MCP)

Memory
Notes carried between steps

The engineer stays outside the loop: sets the goal, grants the tools, approves the result.

Human-in-the-loop: the agent stops for review before a change is accepted.

Agent = LLM + Harness



Same model, better harness, better result. The harness is the part you engineer.

The Harness Lives In The Repo

```
studymatch/
├─ AGENTS.md
├─ product/
│  ├─ product.md
│  └─ stories/US-001-…/spec.md plan.md
├─ docs/design-thinking/
├─.opencode/
│  ├─ agents/  3 agents
│  └─ skills/  6 skills
├─ src/ · e2e/ · supabase/
└─ .github/  PR template
```

CONTEXT	**AGENTS.md · product/ · docs/** What the agent knows before it starts
ROLES	**.opencode/agents/** Product Assistant: what · Planner: how · Coder: build
SKILLS	**.opencode/skills/** Six methods, e.g. product-management
PERMISSIONS	**.opencode/agents/*.md** Front-matter: what each agent may edit and run
CHECKS	**e2e/ · npm scripts** Lint, types, tests, build, Playwright before every PR

If it is not in the repo, the agent does not know it. Every rule is a file you can review and version.

3 Agents In Our Template-Harness ：
- Separate Roles, Skills And Files

AGENT · PRODUCT ASSISTANT

Why and what

- Turns discovery into product framing
- Writes and refines user stories with ACs
- Sets story status: backlog, ready, done
- Classifies feedback for the PO

Skills: product-management, feedback-management

Writes: product.md and stories/<id>/spec.md

Never: code or technical plans

AGENT · PLANNER

How

- Plans exactly one Ready story
- Checks the code before planning
- Maps every AC to a test
- Names risks and human checkpoints

Skills: plan-story

Writes: stories/<id>/plan.md

Never: implements or changes the ACs

AGENT · CODER

Build and verify

- Opens the Issue and a feature branch
- Implements the plan with tests
- Runs the full gate, reports each AC
- Opens the PR, deploys on acceptance

Skills: implement-story, ui-design, deploy

Writes: code, tests and the pull request

Never: edits requirements or merges alone

The Product Owner owns the what and accepts the value. The System Engineer owns the how, the quality gate and the merge.

An Agent Is A Role, A Skill Is A Method

Agent file Who and what

WHERE
.opencode/agents/product-assistant.md

CONTAINS
Role, limits, permissions, hand-off

LOADED
Always, while the agent runs

ANSWERS
Who acts, and what it may touch

Skill How

WHERE
.opencode/skills/product-management/

CONTAINS
Description, steps, template

LOADED
Only when the task needs it

ANSWERS
How the task is done, every time

Keep roles small and methods reusable. One agent can use several skills.

Anatomy Of An Agent File

FILE `.opencode/agents/product-assistant.md`

FRONT-MATTER enforced by the harness

```
---
description: Product Assistant. Why and what.
permission:
  edit: { "product/**": allow, "*": deny }
  bash: deny
---
```

PROMPT guidance for the model

```
You own the what and the why, never the how.
Use the product-management skill for stories.
Write: product.md and stories/<id>/spec.md
Never: code or technical plans.
```

1 Permissions

May edit product/ only. No shell.

2 Role and limits

What and why. Never code or plans.

3 Hand-off

Delivers spec.md for the Planner.

The front-matter is enforced by the harness. The prompt sets the role.

Simplified excerpt. The full file is in the template repo.

Anatomy Of A Skill

FILE .opencode/skills/product-management/SKILL.md

```
---
name: product-management
description: Write user stories with ACs.
  Use when a story is created or made ready.
---
1. Restate the user need in one sentence
2. Write the story and testable ACs
3. Add constraints and out of scope
4. Ask the PO about anything unclear
Template: spec.md
```

1 Description

Decides when the agent loads it

2 Steps

The same method, every time

3 Template

Gives spec.md its four parts

Six skills, three agents. Fix a skill once, and every agent gains.

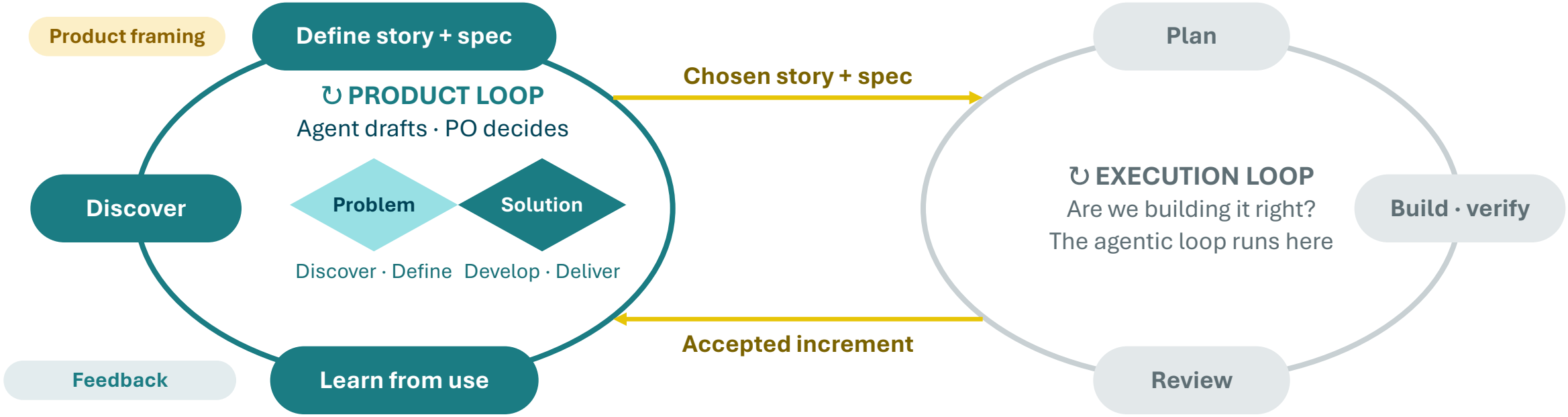
Simplified excerpt. The full file is in the template repo.

Agentic Development: -The Product Loop

Agentic Development

04

Product Loop: Product Framing



Artefacts: product.md · stories/<id>/spec.md
People decide value; the Product Assistant drafts and asks

Execution loop: next section
Planner and Coder build one Ready story

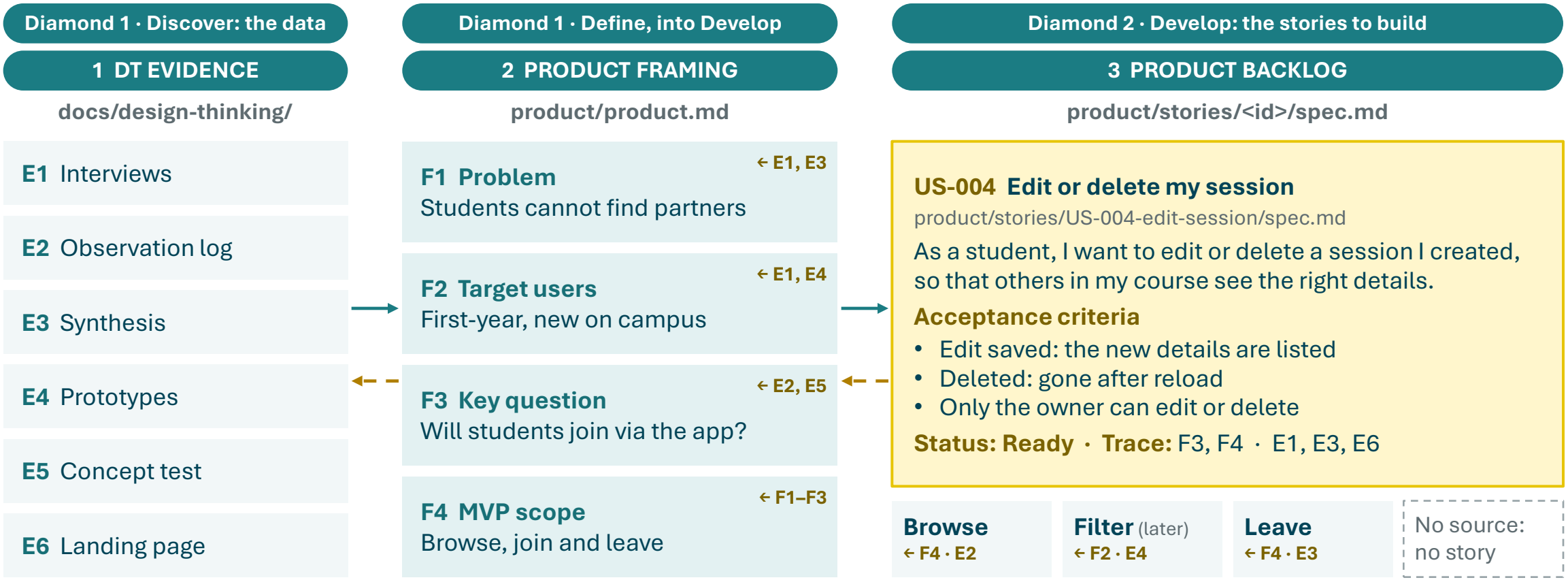
The product loop decides what to build. Agents help at every step, but the PO decides.

Product Loop: Product Framing II

DT PHASE	Diamond 1 · Define: converge on the problem			Diamond 2 · Develop: shape the solution	
	1 Problem	**2 Target users**	**3 Key question**	**4 MVP scope**	**5 Success metric**
AGENT Product Assistant	Clusters the evidence and drafts problem statements	Proposes personas from the interview notes	Turns assumptions into testable hypotheses	Lists what is in and out, and flags risks	Suggests measurable success criteria
PEOPLE Product Owner	**Pick the problem worth solving**	**Confirm who the MVP is for**	**Choose the one question to answer**	**Draw the MVP line**	**Set the target, e.g. 30% join a session in week 1**
DATA	**In: evidence from Discover**		**Out: product/product.md, every statement labelled**		

The Product Assistant drafts, the PO decides. No story is written until the PO has approved the framing.

Product Loop: Product Framing III



One story, one spec.md. It holds the status and traces back to the evidence.

Walkthrough: Product Framing

Product loop · Diamond 1 · Discover and Define, into Develop

1 Copy	2 Frame	3 Ask	4 product.md	5 Backlog
The PO copies the DT documentation into docs/design-thinking/. **Watch:** The agent reads files, not chat.	The Product Assistant reads the evidence and drafts problem, users and scope. **Watch:** Every line traces to a source.	The Product Assistant asks the PO along the way, one question at a time. **Watch:** Answers become decisions.	The Product Assistant writes product.md and the PO approves the framing. **Watch:** Evidence, assumption and decision are labelled.	The Product Assistant creates one folder with a spec.md per derived story. **Watch:** status: backlog, traced to the framing.
You copy	Product Assistant	Together	You approve	Product Assistant

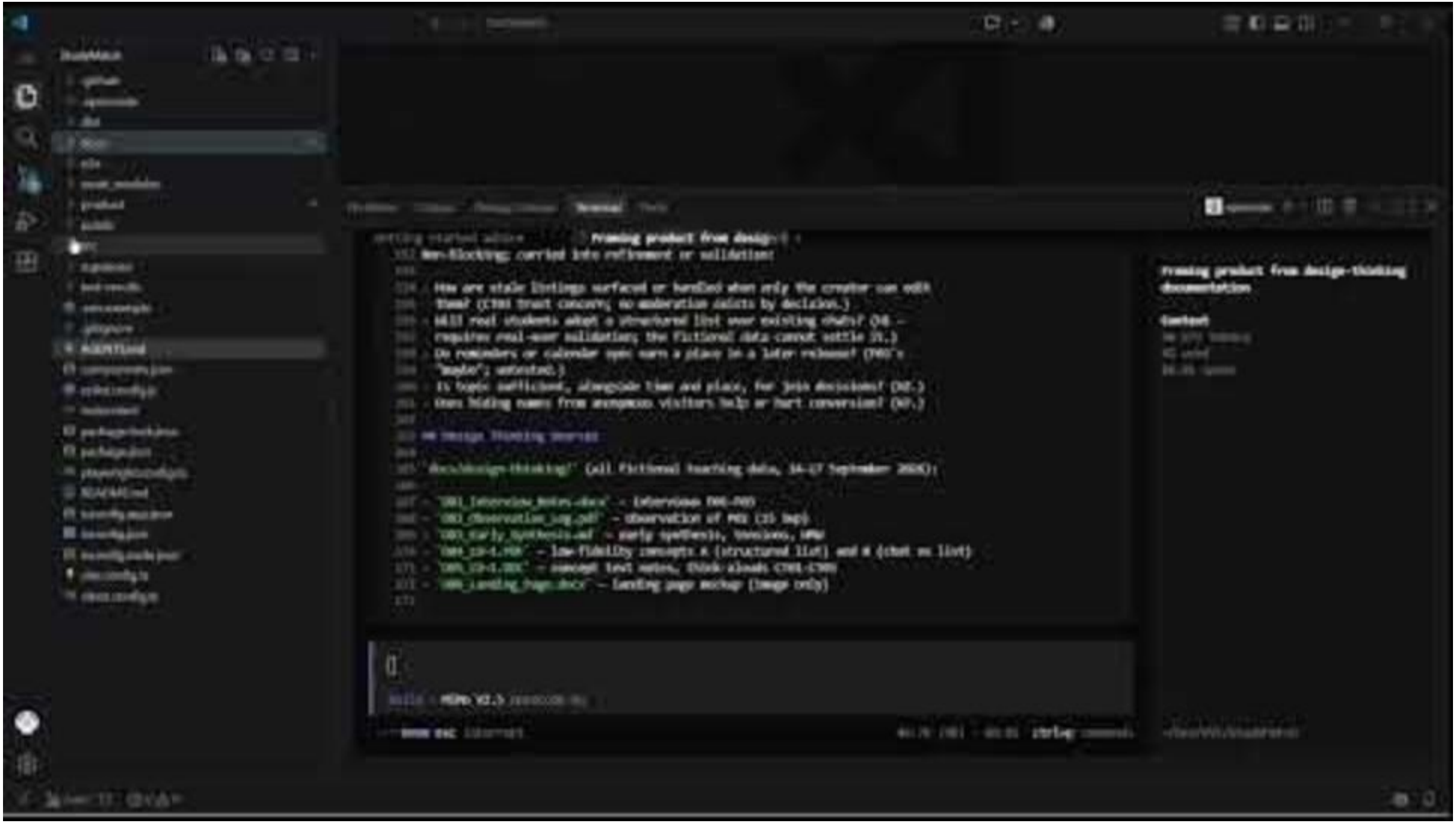
From notes to a Product Backlog: Framing in product.md, one user story per spec.md.

Demo 1

From Discovery To Product Framing

Open: docs/design-thinking/

D1



Product Framing Gave Us The Product Backlog

← From product framing	Scrum terms, and where they live in the repo	
IN SCRUM	**IN OUR REPO**	**WHERE**
Product Backlog	product/stories/, one folder per user story	Framing
Product Backlog Item	One spec.md per user story, status: backlog	Framing
Backlog refinement	The Product Assistant asks, the PO answers	Refinement
Definition of Ready	Five Ready checks pass, then status: ready	Refinement

The Product Backlog exists. Next, refinement makes one item Ready.

Walkthrough: Backlog Item Refinement

- A Story Becomes Ready

Product loop · Diamond 2 · Develop: refine one story → **Execution loop**

1 Pick	2 Draft	3 Ask	4 Check	5 Handover
The PO picks one item from the Product Backlog: a spec.md with status: backlog. **Watch:** One folder, one story.	The Product Assistant drafts observable acceptance criteria. **Watch:** Behaviour, not implementation.	The Product Assistant asks the PO one open question at a time. **Watch:** Each answer goes into spec.md.	The Product Assistant runs the five Ready checks and lists what is out of scope. **Watch:** No open question left.	The Product Assistant sets status: ready. The spec goes to the Planner. **Watch:** The live demo starts from this file.
You pick	**Product Assistant**	**You answer**	**Product Assistant**	**You hand over**

No GitHub yet: the story lives in product/stories/ until the Planner and Coder take it.

One Ready story. Agent: Product Assistant, skill product-management. Next: the Planner.

Demo 2

A Product Backlog Item Becomes Ready

Open: product/stories/<id>/spec.md

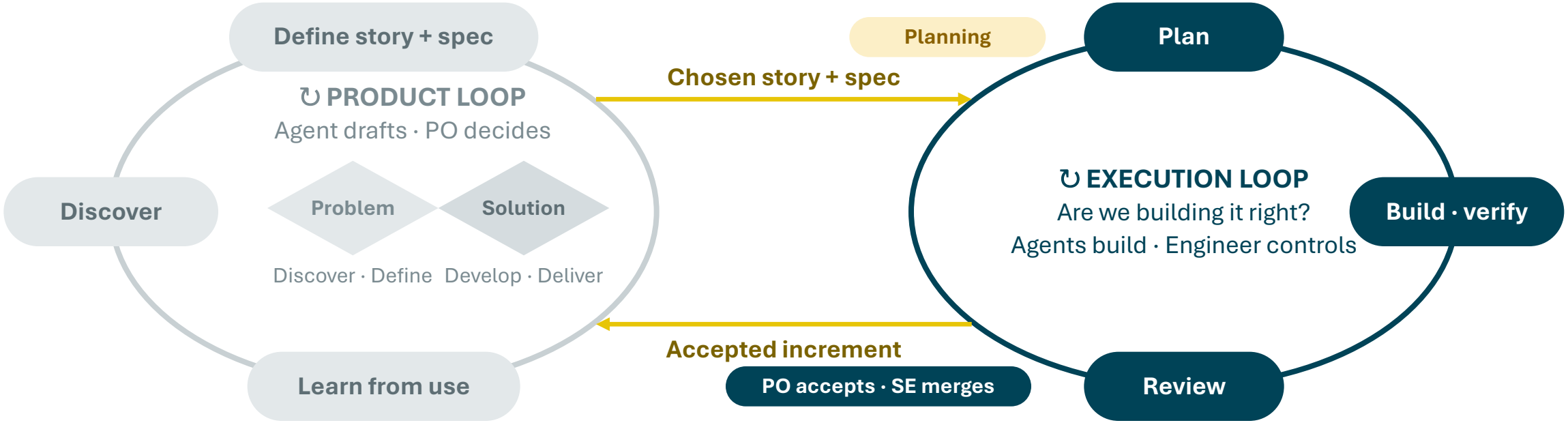
D2



Agentic Development: The Execution Loop

05

Execution Loop: From Ready Story To Increment



The execution loop builds it right. Agents do the work, the System Engineer controls it, the PO accepts it.

Agents Build, Engineer Controls, PO Accepts

LOOP	Execution loop · The Sprint: plan, build and verify one story				→ Product loop
	1 Plan	2 Implement	3 Test	4 Pull request	5 Accept
AGENTS Planner, Coder	Planner reads spec.md and the code, writes the plan	Coder opens a GitHub Issue and a branch, then builds	Coder runs the full gate and checks every AC	Coder self-reviews, opens the PR and stops	On the engineer's go, the Coder deploys
HUMAN Engineer · PO	**Engineer: check the plan fits the story**	Engineer: watch, stop it if it drifts	Engineer: read the AC verdicts	**Engineer reviews the code, PO tries the Preview**	**PO accepts, engineer merges**
OUTPUT	plan.md	Issue + branch	AC: PASS / FAIL	Pull request	Deployed MVP

One Ready story at a time. The Coder stops at the PR; the PO accepts and the System Engineer merges.

The Planner Agent Turns A Story Into A Plan

1 spec.md · why and what

US-004 Edit or delete my session

As a student, I want to edit or delete a session I created, so that others in my course see the right details.

Acceptance criteria

AC1 Edit saved and listed
AC2 Delete gone after reload
AC3 Only the owner can change it

2 plan.md · how

Planner writes only plan.md

- Existing system: facts, not guesses
- Approach and rejected options
- Migration, or the reason for none
- Every AC mapped to a test
- Human checkpoints it cannot fake

3 Implementation sequence

1 Migration and RLS policy
2 Edit form and save
3 Delete, gone after reload
4 Hide actions for others
5 Tests and Playwright journey

The plan holds the how, never the what. It may not weaken, redefine or expand an AC; a gap goes back to the PO.

Demo: Plan And Build
- From Story To Coded Solution

Execution loop · The Sprint: plan, build and verify one story

1 Plan	2 Branch	3 Build	4 Gate
The Planner writes plan.md for US-004. **Watch:** The engineer reviews: facts checked, every AC mapped.	The Coder opens a GitHub feature branch. **Watch:** The branch name carries the story ID.	The Coder implements the plan and writes the tests. **Watch:** Human prerequisites stay with the engineer.	Lint, types, tests, build and Playwright run locally. **Watch:** PASS or FAIL per AC.
Planner	Coder	Coder	Coder

One story, end to end. The agent builds; the plan and the quality gate keep it on the story.

Demo 3

Plan, build and verify US-004

D3

Feedback Goes Back Through The Product Loop

Learn from use | **Product loop · Diamond 1 · Discover and Define again**

USER TEST	FEEDBACK	PRODUCT ASSISTANT	PO DECIDES	RESULT
Students test the MVP On the deployed version	**“Hide Sign In when I’m signed in!”** Logged as feedback	**Classifies it** Usability issue, inside MVP scope	**Accept it** Fits the MVP scope	**New story US-008** Added to the backlog

THE PRODUCT ASSISTANT CLASSIFIES FEEDBACK AS

Defect | **Usability issue** | **Changed requirement** | **New user need** | **Feature idea** | **Invalidated assumption** | **No action**

The outcome is one of four: change an existing story, create a new story, update product.md, or take no action.

Feedback never becomes code directly. It goes through the Product Assistant and a PO decision.

Walkthrough: From Feedback To Merged Change

- Run The Full Procedure Once More

Product loop →

Execution loop · plan, build, review and merge

1 Story	2 Plan	3 Build	4 Close
The Product Assistant classifies the change and writes US-008. **Watch:** The PO approves the story and its ACs.	The Planner writes the plan. The engineer approves it. **Watch:** Every AC mapped to a test.	The Coder branches, builds, tests and opens the PR for the engineer. **Watch:** PASS or FAIL per AC.	The PO tests the Preview and accepts. The engineer merges. **Watch:** spec.md moves to status: done.
PO decides	**Planner**	**Coder**	**PO accepts · SE merges**

One change, end to end. Feedback becomes a story before it becomes code.

Demo 4

From Feedback To A Merged Change

D4

Agentic Development:
-The MVP Playbook

06

The MVP Playbook: Seven Steps, Two Loops

PRODUCT LOOP · The agent drafts, you decide as Product Owner

1 Collect
Gather design-thinking evidence about users and the problem

2 Frame
Who it is for, which problem, which outcome; you decide as PO

3 Story
One spec per story, with testable acceptance criteria (ACn)

4 Ready
Refine one question at a time until the story is Ready

EXECUTION LOOP · The agent plans and builds, you control as System Engineer

5 Plan
The agent proposes a technical plan; you approve it as engineer

6 Build and verify
The agent builds the smallest slice and checks every ACn

7 Accept and learn
Accept PASS or FAIL per ACn as PO, release, test with users

Feedback → new or revised stories. Same seven steps in both tracks. Only the tools in the execution loop differ.

Your Take-Home Harness: OpenCode Track

```
your-mvp/
├─ AGENTS.md
├─ product/
│  ├─ product.md
│  └─ stories/US-XXX-<slug>/
│     spec.md · plan.md · verification.md
├─ docs/
│  ├─ design-thinking/
│  └─ architecture.md · ui-guidelines.md
├─ .opencode/  agents/ · skills/
├─ src/ · e2e/
├─ supabase/migrations/
└─ .github/  PR template
```

product/ PRODUCT LOOP
product.md holds the framing. One folder per story is the backlog: no backlog.md, no Issues.

.opencode/ AGENTS AND SKILLS
Product Assistant, Planner and Coder, with six skills.

stories/<id>/plan.md · verification.md EXECUTION LOOP
The Planner's plan you approve as engineer, and the Coder's evidence you read before you accept as PO.

src/ · e2e/ · supabase/ · .github/ BUILD AND CHECKS
npm run verify: lint, types, tests, build and E2E, before the PR and again before merge.

One repo, both loops in OpenCode. The app code and its checks live next to the specs. Template: MF-MyMVP-Template.

Your Take-Home Harness: Lovable Track

```
your-mvp/
├─ AGENTS.md · opencode.json
├─ product/
│  ├─ product.md
│  └─ stories/US-XXX-<slug>/
│     spec.md · acceptance.md
├─ docs/
│  └─ design-thinking/
├─ .opencode/  agents/ · skills/
└─ lovable/
   ├─ skills/
   └─ workspace-knowledge.md

App code lives in the Lovable project
```

product/ PRODUCT LOOP · SHARED

product.md and one folder per story: spec.md goes out, acceptance.md comes back. Plan and verification stay in Lovable.

.opencode/ PRODUCT LOOP IN OPENCODE

Product Assistant drafts the specs. /handover sends spec.md to Lovable, /return brings acceptance.md back.

lovable/skills/ EXECUTION LOOP IN LOVABLE

/harness-init, /build-story, /accept-story and /publish, imported once per workspace.

lovable/workspace-knowledge.md RULES FOR LOVABLE

Pasted into workspace knowledge, so Lovable follows the same rules. It builds and checks each ACn on the preview.

A product repo in OpenCode, the app in Lovable. Template: MF-MyMVP-Lovable-Template (pre-release v0.1.0).

Take it home !

Scan the QR code for all documentation, the video lecture, demos and harnesses for both tracks.

Per Christian Engdal
Asst. Prof., Western Norway University of Applied Sciences
per.christian.engdal@hvl.no
+47 98 26 93 01

https://pcehvl.github.io/MakersForge-2026

L.G.T.M: https://www.youtube.com/watch?v=3TNpOD6bov8